CSCI 2406: Topic 02

RISC Principles and ARM Architecture

RISC Design Principles, AArch64 Architectural Overview,
Registers and Programmer-Visible State, and Instruction Classes

Computer Organization & Architecture | AArch64 Reference

Topic 2 Outline

Topic Objective

Examine the design principles commonly associated with RISC architectures and show how those principles appear concretely in AArch64.

- **Section 1**: RISC Design Principles
- **Section 2**: AArch64 Architectural Overview
- **Section 3**: Registers and Programmer-Visible State
- **Section 4**: Core AArch64 Instruction Classes
- **Section 5**: Summary and Self-Test

Section 01

RISC Design Principles

What Does RISC Mean?

Reduced Instruction Set Computer

RISC is a design philosophy emphasizing **regularity**, **simple instruction forms**, and a clean separation between computation and memory access.

- RISC does **not** simply mean "a processor with very few instructions."
- "Reduced" is better understood as **reduced instruction complexity and irregularity**.
- Arithmetic and logical operations are primarily register-based.
- Load and store instructions transfer data between registers and memory.

Foundational RISC Design Principles

Instruction Regularity

- Regular instruction formats.
- Consistent operand fields.
- Predictable instruction alignment.

Register-Centered Computation

- Arithmetic uses registers.
- Memory access uses loads and stores.
- Regularity simplifies decoding.

Implementation Benefit

These characteristics support regular instruction fetch, decode, and pipelined implementation.

The Load/Store Architectural Model

Separation of Memory Access and Computation

In a load/store architecture such as AArch64, arithmetic and logical instructions operate on registers. Memory is accessed using dedicated **load** and **store** instructions.

Data Processing

- Operands are registers.
- Results are written to registers.

```
add x1, x2, x3
```

Memory Access

- Loads read memory into registers.
- Stores write registers to memory.

```
ldr x1, [x0]
```

Load–Compute–Store in AArch64

Suppose the address in `x0` identifies a 64-bit value in memory that must be increased by 5.

```
ldr x1, [x0]                    // load memory value into x1
```

```
add x1, x1, #5                  // compute using register operands
```

```
str x1, [x0]                    // store result back to memory
```

Architectural Rule

The `add` instruction does not directly read or modify a memory operand.

Visualizing Load/Store Separation

Takeaway

Loads and stores connect memory to registers; arithmetic instructions operate on register operands.

RISC and CISC: Different Design Traditions

Characteristic	CISC Tradition	RISC Tradition
Instruction format	Often variable length	Typically more regular
Memory operands	Some instructions combine memory access and computation	Load/store separation
Addressing	Often many addressing forms	Comparatively regular forms
Decode	Can require more complex parsing	Regular fields simplify decode
Example	x86-64	AArch64, RISC-V

RISC and CISC are architectural traditions, not simple measures of modern processor complexity.

AArch64 vs. x86-64: Instruction Length

AArch64 / A64

32 bits = 4 bytes

- Fixed instruction width.
- Instructions are 4-byte aligned.
- Boundaries are known immediately.

x86-64

1 to 15 bytes

- Variable instruction length.
- Length depends on the encoding.
- Boundaries must be discovered.

What a 16-Byte Fetch Might Contain

A64

`4B | 4B | 4B | 4B`

Exactly four instructions.

x86-64

`1B | 4B | 2B | 7B | 2B`

Five instructions of different lengths.

A64 starts with known boundaries; x86-64 must determine them.

Why x86 Instruction Decode Is More Irregular

An x86-64 instruction may contain several optional or variable-size components.

Some fields are optional; some fields vary in size; the total instruction may be 1–15 bytes.

x86 Front-End Work

- Find instruction boundaries.
- Interpret prefixes and opcode bytes.
- Decode optional addressing fields.
- Handle displacement and immediate lengths.

A64 Front-End Starting Point

Exactly 32 bits
One A64 Instruction

The decoder already knows where the instruction begins and ends.

What Does “Reduced” Buy the Hardware?

Reduced Front-End Irregularity

A regular ISA can reduce some of the circuitry required to **find**, **align**, and **decode** instructions.

Potential Hardware Benefits

- Simpler instruction-boundary detection.
- More regular decode paths.
- Less handling of legacy encoding cases.
- Potentially lower front-end area and switching activity.

But Not a Simple CPU

Modern AArch64 cores may still contain:

- out-of-order execution
- register renaming
- branch predictors
- large caches
- SIMD/vector units

Why AArch64 Fits Mobile and Energy-Constrained Systems

AArch64 provides a regular architectural foundation that works well under tight power and thermal budgets.

Architectural Characteristics

- Fixed 32-bit A64 instructions.
- Regular instruction alignment.
- Explicit load/store memory access.
- Large register set.
- Comparatively regular decoding.

System-Level Advantages

- Efficient processor front ends.
- Strong performance-per-watt designs.
- Highly integrated ARM-based SoCs.
- Long-established mobile ecosystem.
- Hardware can be specialized for a tight energy budget.

Important Qualification

AArch64 does not guarantee lower power. Energy consumption depends on the complete microarchitecture, semiconductor process, clock frequency, voltage, caches, and workload.

Same Operation, Different ISA Style

Suppose a 64-bit memory value must be increased by 5.

x86-64 Style

Some x86-64 instructions permit memory operands directly:

```
add qword ptr
[rax], 5
```

One instruction describes both memory access and arithmetic.

AArch64 Load/Store Style

```
ldr x1, [x0]
add x1, x1, #5
str x1, [x0]
```

Memory transfer and arithmetic are represented by separate instructions.

Section 02

AArch64 Architectural Overview

ARM and AArch64

ARM Architecture

ARM defines a family of instruction set architectures used in embedded systems, mobile computing, desktop systems, and servers.

- **AArch64** is the 64-bit execution state introduced with ARMv8-A.
- It provides a modern 64-bit programmer-visible register model.
- It follows a load/store design with regular instruction formats.
- Different processors can implement AArch64 using different microarchitectures.

AArch64 and A64 Terminology

AArch64

- The architecture / execution state.
- Defines programmer-visible state.
- Includes registers, memory behavior, and instruction semantics.

A64

- The instruction set encoding.
- Used while executing in AArch64 state.
- A64 instructions are fixed 32-bit words.

Course Terminology

Use **AArch64** by default. Use **A64** only when referring specifically to the instruction set encoding.

Key Characteristics of AArch64

Instruction Model

- Load/store architecture.
- A64 instructions are 32 bits wide.
- Instructions are 4-byte aligned.
- Register fields are regular.

Register Model

- 31 general-purpose registers.
- 64-bit `x` register views.
- 32-bit `w` register views.
- Special roles for `sp`, `xzr`, PC state, and NZCV.

A64 Instruction Format Characteristics

Fixed-Width Encoding

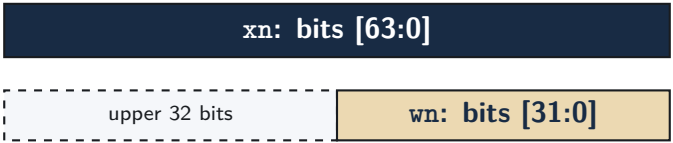
Every instruction in the A64 instruction set occupies exactly **32 bits**.

- Fixed width simplifies instruction alignment and initial decode.
- Exact bit-field layouts vary by instruction class.

Detailed machine-code field layouts are covered later in the instruction-encoding topic.

Dual Register Views: `X` and `W`

Each general-purpose register can be accessed using either a 64-bit or 32-bit architectural view.



32-Bit Write Rule

Writing to a `wn` destination automatically clears bits [63:32] of the corresponding `xn` register.

Example: 32-Bit Write Clears Upper Bits

Instruction

```
add w3, w1, w2
```

- The operation uses the lower 32-bit views: `w1` and `w2`.
- The 32-bit result is written into `w3`.
- The upper half of `x3` is cleared automatically.

```
x3[63:32] <- 0
x3[31:0] <- result
```

Section 03

Registers and Programmer-Visible State

AArch64 General-Purpose Registers

Register Set

AArch64 provides **31 general-purpose registers**, named `x0` through `x30`. Each register is 64 bits wide.

64-Bit View

`x0`–`x30`

- Each `xn` holds 64 bits.
- Used for integer values, addresses, and general computation.

32-Bit View

`w0`–`w30`

- Each `wn` refers to bits [31:0] of `xn`.
- Writing `wn` clears bits [63:32].

Important

The ISA treats x0–x30 as general-purpose registers. Software conventions assign additional

The Zero Register

`xzr` / `wzr`

The zero register has two special behaviors: **reads return zero** and **writes are discarded**.

Use as a Zero Operand

```
mov x0, xzr
```

- Reading `xzr` produces the value `0`.
- Therefore, `x0` receives zero.

Use as a Discard Destination

```
cmp x1, x2
```

- `cmp` is an alias for `subs xzr, x1, x2`.
- The subtraction result is discarded, while NZCV flags are updated.

Key Idea

`xzr` behaves like a permanent zero value when read, and like a discard destination when

The Stack Pointer

sp

The stack pointer identifies the current stack location and has dedicated architectural semantics.

- Used by stack-related load/store and arithmetic forms.
- Represents a memory address, not a normal scratch register.
- Software conventions impose alignment and usage requirements.
- Stack behavior is studied more deeply with procedures and function calls.

```
stp x29, x30, [sp, #-16]!
```

Encoding Note: Register Number 31

Context-Dependent Meaning

In several A64 instruction formats, register encoding value 31 has special meaning.

Often Means Zero Register

`xzr` / `wzr`

Common in data-processing contexts.

Sometimes Means Stack Pointer

`sp` / `wsp`

Common in stack and memory-addressing contexts.

The instruction definition determines the meaning.

The Program Counter

Program Counter State

The program counter identifies the instruction address associated with the current flow of execution.

- Sequential execution advances to the next instruction.
- Branches and calls redirect execution to another address.
- Unlike AArch32 `r15`, the AArch64 program counter is **not** an ordinary general-purpose register.
- AArch64 arithmetic instructions cannot use `pc` as a normal register operand.

Condition Flags: NZCV

Some AArch64 instructions update four condition flags used by later operations.

- **N**: result has its sign bit set.
- **Z**: result is zero.
- **C**: carry-out for addition; no-borrow for subtraction.
- **V**: signed arithmetic overflow.

Programmer-Visible State: Summary

State	Architectural Role
`x0`–`x30`	31 general-purpose 64-bit registers
`w0`–`w30`	Lower 32-bit views of corresponding `x` registers
`xzr`/`wzr`	Zero operand; writes are discarded
`sp`	Stack pointer with dedicated semantics
Program Counter	Identifies control-flow position; not an ordinary GPR
NZCV	Condition flags used by flag-setting and conditional operations

Section 04

Core AArch64 Instruction Classes

Three Core Instruction Classes

For this course, core integer instructions are grouped into three broad functional classes.

Data Processing	Memory Access	Control Flow
• arithmetic	• loads	• branches
• logic	• stores	• calls
• comparison	• transfer widths	• returns
`add`, `sub`	`ldr`, `str`	`b`, `b.eq`
`and`, `cmp`	`ldp`, `stp`	`bl`, `ret`

Class 1: Data Processing

Data-processing instructions perform computation on register values.

Instruction	Operation	Effect
`add x3, x1, x2`	Addition	`x3 <- x1 + x2`
`sub x3, x1, x2`	Subtraction	`x3 <- x1 - x2`
`and x0, x1, x2`	Bitwise AND	`x0 <- x1 & x2`
`orr x0, x1, x2`	Bitwise OR	`x0 <- x1 | x2`

Key Characteristic

The operands are registers; these instructions do not directly read or write a memory operand.

Class 2: Memory Access

Memory instructions transfer data between registers and memory.

```
ldr x1, [x2]                // load 64 bits from memory into x1
```

```
str x5, [x3]                // store 64 bits from x5 to memory
```

```
ldr x1, [x2, #8]            // base address x2 plus byte offset 8
```

Bracket Notation

`[x2]` means: access memory using the address contained in `x2`.

Class 3: Control Flow

Control-flow instructions change which instruction executes next.

Unconditional

- `b target`: branch
- `bl function`: call
- `ret`: return

Conditional

- `b.eq target`: branch if equal
- `b.ne target`: branch if not equal
- Conditional branches may test NZCV flags.

Comparison and Branching

Common Pattern

```
cmp x0, x1
b.eq equal
```

- `cmp` compares two register values.
- It updates condition flags rather than storing a data result.
- `b.eq` uses those flags to decide whether to branch.

How the Instruction Classes Work Together

Consider:

```
*p = *p + 5
```

where `x0` contains the address represented by `p`.

Instruction	Class	Purpose
`ldr x1, [x0]`	Memory Access	Move the value from memory into a register
`add x1, x1, #5`	Data Processing	Compute the new value
`str x1, [x0]`	Memory Access	Write the result back to memory

RISC Connection

The sequence separates memory transfer from register computation.

Instruction Classification Exercise

Classify each instruction as **Data Processing**, **Memory Access**, or **Control Flow**.

Instruction	Your Classification
`add x3, x1, x2`	
`ldr x0, [x4]`	
`str w5, [x6]`	
`cmp x1, x2`	
`b.eq target`	
`bl function`	

Section 05

Summary & Self-Test

Topic 2 Summary

- **RISC Design**: emphasizes reduced instruction complexity, regular formats, register-based computation, and explicit load/store memory access.
- **Instruction Regularity**: every A64 instruction is 32 bits, while x86-64 instructions may range from 1 to 15 bytes.
- **AArch64**: ARM's 64-bit architectural execution state.
- **Programmer-Visible State**: `x0`–`x30`, `sp`, `xzr`/`wzr`, PC state, and NZCV.
- **Instruction Classes**: Data Processing, Memory Access, and Control Flow.

Self-Test 1/3

1. Why does “reduced” in RISC not simply mean fewer instructions?
2. What does it mean to say that AArch64 is a load/store architecture?
3. Why does fixed-width instruction encoding simplify instruction fetch and decode?
4. How does A64 instruction length differ from x86-64 instruction length?

Self-Test 2/3

5. What happens to bits [63:32] of `x5` after an instruction writes to `w5`?
6. How many general-purpose registers are visible in AArch64?
7. What are the roles of `xzr`, `sp`, and the program counter?
8. What do the `N`, `Z`, `C`, and `V` condition flags represent?

Self-Test 3/3

9. What are the three core instruction classes used in this course?
10. Classify each instruction: `add`, `ldr`, `str`, `cmp`, and `b.eq`.

Challenge

Explain why fixed 32-bit A64 instructions can require less instruction-boundary logic than variable-length x86-64 instructions.

Wrap-Up and Next Steps

Topic 3: Machine-Level Data Representation

The next lecture examines how bit patterns manipulated by AArch64 instructions represent numerical values in registers and memory.

- Binary and hexadecimal representation
- Signed and unsigned integers
- Two's-complement representation
- Endianness and data alignment

Topic 2 established **what the AArch64 programmer can see**; Topic 3 examines **how data inside that visible state is represented**.